

AIML & DSA Practice Question Bank

50-Question Master Prep Reduced to 25 Core Test Questions

Target: Class Test Preparation | **Includes:** 10 Core Exam Debugging MCQs (★ Marked) + 15 Conceptual Practice MCQs + Coding Reference

Instructions for Students: Questions 1–10 are directly taken from your syllabus exam paper and are **marked with ★ (IMPORTANT CORE)** as primary must-know items for your class test. Questions 11–25 build essential conceptual depth across Data Structures & Algorithms.

Section A: 10 Important Core Debugging MCQs (★ Priority Focus)

Question 1: Kth Largest Element in BST

★ IMPORTANT CORE

The code below is intended to find the Kth Largest element in a Binary Search Tree (BST) but fails on several test cases. Identify the bug:

```
def kthLargest(root, k):
    result = []
    def inorder(node):
        if not node:
            return
        inorder(node.left)
        result.append(node.val)
        inorder(node.right)
    inorder(root)
    return result[k - 1]
```

- A) The recursion never visits the right subtree.
- B) It performs a standard (ascending) inorder traversal, so `result[k-1]` returns the Kth smallest element instead of the Kth largest.
- C) The base case condition `if not node` is incorrect.
- D) The function does not handle duplicate values in the BST.

Question 2: Inorder Predecessor in BST

★ IMPORTANT CORE

This function should return the inorder predecessor of key in a BST but gives wrong results. What is the bug?

```
def inorderPredecessor(root, key):
    pred = None
    while root:
        if root.val >= key:
            root = root.right    # move here
        else:
            pred = root
            root = root.right
    return pred
```

- A) pred should be updated inside the first branch, not the else-branch.
- B) When `root.val >= key`, the search must move left (`root = root.left`) to look for a smaller predecessor; moving right is the bug.
- C) The while loop condition should be `while root.left:`.
- D) The function should be written recursively instead of iteratively.

Question 3: Lowest Common Ancestor (LCA) in BST

★ IMPORTANT CORE

The LCA function below returns an incorrect ancestor for some inputs. Identify the bug:

```
def lowestCommonAncestor(root, p, q):
    while root:
        if p.val < root.val and q.val < root.val:
            root = root.left
        elif p.val > root.val and q.val > root.val:
            root = root.left
        else:
            return root
    return None
```

- A) The first branch should move to `root.right` instead of `root.left`.
- B) When both `p.val` and `q.val` are greater than `root.val`, the search should move to `root.right`, not `root.left`.
- C) The else branch should return `None` instead of `root`.
- D) The while loop should be a for loop.

Question 4: Sorted Array to Height-Balanced BST

★ IMPORTANT CORE

This function should convert a sorted array into a height-balanced BST but produces an incorrect or infinite structure. What is the bug?

```
def sortedArrayToBST(nums):  
    if not nums:  
        return None  
    mid = len(nums) // 2  
    root = TreeNode(nums[mid])  
    root.left = sortedArrayToBST(nums[mid:])  
    root.right = sortedArrayToBST(nums[mid + 1:])  
    return root
```

- A) `root.right` should use `nums[mid:]` instead of `nums[mid + 1:]`.
- B) `root.left` should be built from `nums[:mid]`, not `nums[mid:]` — using `nums[mid:]` causes the same middle element to be reused, leading to infinite recursion.
- C) `TreeNode(nums[mid])` should be `TreeNode(nums[0])`.
- D) The base case should check `len(nums) == 1` instead of `not nums`.

Question 5: Min-Heap Kth Largest Element

★ IMPORTANT CORE

This min-heap based solution for the Kth Largest Element sometimes returns the wrong value. What is the bug?

```
import heapq  
def findKthLargest(nums, k):  
    heap = nums[:k]  
    heapq.heapify(heap)  
    for num in nums[k:]:  
        if num > heap[0]:  
            heapq.heappush(heap, num)  
    return heap[0]
```

- A) It should use a max-heap instead of a min-heap.
- B) After `heappush`, the code never pops the smallest element (`heappop`), so the heap grows beyond size `k` and `heap[0]` no longer represents the Kth largest.
- C) `heapq.heapify()` modifies the original `nums` list, which is disallowed.
- D) The comparison should be `num >= heap[0]`.

Question 6: Top K Frequent Elements

★ IMPORTANT CORE

This function is intended to return the K most frequent elements but returns incorrect elements. What is the bug?

```
from collections import Counter
import heapq
def topKFrequent(nums, k):
    count = Counter(nums)
    heap = []
    for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
    return [num for freq, num in heap[:k]]
```

- A) Counter cannot count elements of a list.
- B) `heapq.heappush` should not be used with tuples.
- C) A Python heap list is only partially ordered (only root is guaranteed smallest); slicing `heap[:k]` does not give the k largest-frequency elements.
- D) The tuple order should be `(num, freq)` instead of `(freq, num)`.

Question 7: Brute-Force Two Sum

★ IMPORTANT CORE

This brute-force Two Sum implementation occasionally returns an invalid answer. What is the bug?

```
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

- A) It doesn't handle negative numbers correctly.
- B) The inner loop allows `j == i`, so the same element could be paired with itself even though each element may only be used once.
- C) It should return the values instead of the indices.
- D) The time complexity is $O(n \log n)$, which is too slow.

Question 8: Group Anagrams

★ IMPORTANT CORE

This function should group anagrams together but instead puts every string in its own separate group. What is the bug?

```
def groupAnagrams(strs):
    groups = {}
    for s in strs:
        key = s
        groups.setdefault(key, []).append(s)
    return list(groups.values())
```

- A) `setdefault()` is being used incorrectly.
- B) The key used to group strings should be a canonical form such as `' '.join(sorted(s))`, not the raw string `s` itself.
- C) The function should return `groups.keys()` instead of `groups.values()`.
- D) `strs` should be sorted before entering the loop.

Question 9: Longest Consecutive Sequence

★ IMPORTANT CORE

This solution gives correct results but does not guarantee $O(n)$ worst-case time complexity. What optimization is missing?

```
def longestConsecutive(nums):
    num_set = set(nums)
    longest = 0
    for num in num_set:
        length = 1
        current = num
        while current + 1 in num_set:
            current += 1
            length += 1
        longest = max(longest, length)
    return longest
```

- A) It should sort the array first, which would make it $O(n)$.
- B) It should check if `num - 1 not in num_set` before starting to count, so a sequence is only built from its true starting element, avoiding redundant $O(n)$ scans.
- C) It should use a list instead of a set for $O(1)$ lookups.
- D) The while loop condition should check `current - 1` instead of `current + 1`.

Question 10: Task Scheduler

★ IMPORTANT CORE

This implementation of Task Scheduler gives wrong (too-small) results when there are many distinct tasks and cooling interval n is small. What is missing?

```
from collections import Counter
def leastInterval(tasks, n):
    counts = Counter(tasks)
    max_count = max(counts.values())
    max_count_tasks = list(counts.values()).count(max_count)
    return (max_count - 1) * (n + 1) + max_count_tasks
```

- A) The result should be `max(len(tasks), (max_count - 1) * (n + 1) + max_count_tasks)` — the formula must be bounded below by total task count when idle slots aren't required.
- B) Counter cannot count elements of a list of strings.
- C) `max_count` should be the minimum frequency, not the maximum.
- D) `n + 1` should be replaced with just `n`.

Section B: 15 Conceptual & Algorithmic Practice MCQs

Question 11: Two-Heap Stream Median Concept

PRACTICE

In the two-heap approach for finding the median of a data stream, how are $O(1)$ time median queries achieved?

- A) By searching the single min-heap in $O(1)$ time.
- B) By balancing sizes of a Max-Heap (lower half) and Min-Heap (upper half), making median available directly at the heap roots.
- C) By sorting both heaps on every insertion.
- D) By performing BFS traversal across both heaps.

Question 12: Merge K Sorted Lists Time Complexity

PRACTICE

When merging K sorted lists with a total of N elements using a Min-Heap of size K , what is the overall time complexity?

- A) $O(N \cdot K)$
- B) $O(N \log K)$
- C) $O(K \log N)$
- D) $O(N^2)$

Question 13: Worst-Case BST Search Time

PRACTICE

What is the worst-case time complexity of searching for an element in an unbalanced (skewed) Binary Search Tree with N nodes?

- A) $O(1)$
- B) $O(\log N)$
- C) $O(N)$
- D) $O(N \log N)$

Question 14: BST Traversal Order

PRACTICE

Which depth-first tree traversal technique always visits node values of a Binary Search Tree in strictly ascending order?

- A) Preorder Traversal
- B) Postorder Traversal
- C) Inorder Traversal
- D) Level-order Traversal

Question 15: Python Heap Nature

PRACTICE

In Python's built-in `heapq` module, what type of heap structure is implemented by default?

- A) Max-Heap
- B) Min-Heap
- C) Fibonacci Heap
- D) Binary Search Tree

Question 16: Max-Heap Simulation in Python

PRACTICE

How do programmers standardly simulate a Max-Heap using Python's default `heapq` library?

- A) By passing `reverse=True` to `heapq.heappush()`.
- B) By multiplying values by `-1` when inserting and extracting.
- C) By calling `heapq.max_heapify()` before popping elements.
- D) By converting the list to a set before operations.

Question 17: Hash Map Space Complexity

PRACTICE

What is the auxiliary space complexity of storing N key-value pairs in a Hash Map?

- A) $O(1)$
- B) $O(\log N)$
- C) $O(N)$
- D) $O(N^2)$

Question 18: Constant Time Lookup

PRACTICE

Which data structure guarantees average $O(1)$ time complexity for element lookup and insertion?

- A) Sorted Array with Binary Search
- B) Hash Set / Hash Map
- C) Singly Linked List
- D) Binary Heap

Question 19: Divide & Conquer Merging

PRACTICE

What is the primary efficiency advantage of using Divide-and-Conquer when merging K sorted lists compared to sequential standard merging?

- A) Reduces overall space overhead from $O(N)$ to $O(1)$.
- B) Reduces total merge passes by combining lists pairwise in $\log(K)$ levels instead of K levels.
- C) Converts linked lists directly into dynamic arrays.
- D) Reduces time complexity to constant time $O(1)$.

Question 20: Tree Height Bounds

[PRACTICE](#)

In a binary tree containing N nodes, what is the maximum possible height (number of edges on the longest path from root to leaf)?

- A) $\log_2(N)$
- B) $N - 1$
- C) $N / 2$
- D) $2N$

Question 21: Optimal Two-Sum Mapping

[PRACTICE](#)

When solving the Two-Sum problem in a single pass with a Hash Map, what key-value pair representation should be stored?

- A) Key: Array Index, Value: Number Value
- B) Key: Number Value, Value: Array Index
- C) Key: Target Sum, Value: Difference
- D) Key: Number Value, Value: Frequency

Question 22: BST Lowest Common Ancestor Property

[PRACTICE](#)

In a Binary Search Tree, what key mathematical property defines the Lowest Common Ancestor (LCA) node X for two target nodes p and q ?

- A) X must be equal to the root of the tree.
- B) X is the first node during top-down traversal where p and q split into left and right subtrees (or one value matches $X.val$).
- C) X is always the leaf node closest to p .
- D) $X.val$ must equal $p.val + q.val$.

Question 23: Array Slicing Memory Impact

[PRACTICE](#)

Why is array slicing (e.g., `nums[mid:]`) considered suboptimal inside recursive functions?

- A) Slice operations create shallow/deep copies of sub-lists, adding extra $O(N)$ space and time overhead per step.
- B) Slicing changes numerical element values inside the original array.
- C) Slicing converts Python lists into sets automatically.
- D) Slicing prevents the base case from ever triggering.

Question 24: Space-Efficient Recursion Design

[PRACTICE](#)

How can array recursion (like binary tree building) be designed in $O(1)$ auxiliary space without slicing arrays?

- A) Pass left and right integer index boundaries as parameters instead of slicing the array.
- B) Convert the input list into a tuple before passing it recursively.
- C) Convert the array into a linked list first.
- D) Store all elements inside a global dictionary.

Question 25: Heap Insertion Complexity

PRACTICE

What is the worst-case time complexity of inserting a single element into a binary heap containing N elements?

- A) $O(1)$
- B) $O(\log N)$
- C) $O(N)$
- D) $O(N \log N)$

Section C: Exam Paper Coding Reference Questions (30 Marks)

Q1. Merge K Sorted Lists (15 Marks)

You are given an array of k linked lists, each sorted in ascending order. Merge all the linked lists into one sorted linked list and return its head. Solution must achieve $O(N \log k)$ time complexity using Heap or Divide-and-Conquer approach.

Expected Complexity: Time: $O(N \log k)$, Space: $O(k)$ or better.

Q2. Find Median from Data Stream (15 Marks)

Design a data structure supporting `addNum(int num)` and `findMedian()`. Implement `MedianFinder` class using a two-heap system (Max-Heap for lower half, Min-Heap for upper half).

Expected Complexity: `addNum`: $O(\log n)$, `findMedian`: $O(1)$.

Section D: Answer Key & Comprehensive Explanations

Q#	Type	Ans	Explanation & Core Takeaway
★ Q1	CORE	B	Standard inorder traversal (Left-Root-Right) produces values in ascending order. Thus <code>result[k-1]</code> gives the k -th <i>smallest</i> . To find k -th largest, use reverse inorder (Right-Root-Left) or access <code>result[-k]</code> .
★ Q2	CORE	B	When <code>root.val >= key</code> , the target predecessor must reside in the left subtree (smaller values). Moving right searches larger values, which is the bug.
★ Q3	CORE	B	When both <code>p.val</code> and <code>q.val</code> are greater than <code>root.val</code> , both targets lie in the right child. The code incorrectly executes <code>root = root.left</code> .
★ Q4	CORE	B	Slicing <code>nums[mid:]</code> for the left child includes <code>mid</code> , causing the middle element to be re-processed continuously, creating an infinite recursion loop. Left child must use <code>nums[:mid]</code> .
★ Q5	CORE	B	When pushing a new element into a min-heap tracking top- k , <code>heappop(heap)</code> must be called immediately after <code>heappush</code> to maintain heap size at exactly k .
★ Q6	CORE	C	A Python heap list is only guaranteed to have its minimum at <code>heap[0]</code> . Slicing <code>heap[:k]</code> takes arbitrary elements from array representation rather than heap-popped top elements.

Q#	Type	Ans	Explanation & Core Takeaway
★ Q7	CORE	B	Inner loop starting from $j = 0$ allows $j == i$. An element can add to itself (e.g., target 6, array [3,4], index 0 pairs with index 0). Inner loop should start from $i + 1$.
★ Q8	CORE	B	Using raw string <code>key = s</code> creates distinct key entries for every un-identical word. Standardizing key to <code>' '.join(sorted(s))</code> groups anagrams under a shared signature.
★ Q9	CORE	B	Checking <code>if num - 1 not in num_set</code> ensures sequence counting starts only at sequence beginnings, avoiding quadratic re-scanning of intermediate values.
★ Q10	CORE	A	When there are many distinct tasks relative to cooling interval n , no idle slots are required. The minimum total CPU units required can never be less than <code>len(tasks)</code> .
Q11	PRACTICE	B	Max-heap stores lower half, Min-heap stores upper half. Median is average of roots (even length) or top of max-heap (odd length) in $O(1)$ time.
Q12	PRACTICE	B	Heap maintains head nodes of K lists. Each of N total elements is pushed/popped in $O(\log K)$ time, giving $O(N \log K)$ overall time.
Q13	PRACTICE	C	An unbalanced BST degenerates into a single linear linked list, taking $O(N)$ time for search operations.
Q14	PRACTICE	C	Inorder traversal (Left Subtree $\rightarrow$ Root $\rightarrow$ Right Subtree) visits elements in monotonically increasing order in a BST.
Q15	PRACTICE	B	Python's standard <code>heapq</code> module provides a Min-Heap implementation by default.
Q16	PRACTICE	B	Inverting sign of numeric values (multiplying by -1) flips priority order, effectively creating a Max-Heap using Min-Heap primitives.
Q17	PRACTICE	C	Storing N distinct elements inside a hash table requires linear auxiliary space $O(N)$.
Q18	PRACTICE	B	Hash Sets/Maps utilize hashing functions to achieve average $O(1)$ time for search, insertion, and deletion.
Q19	PRACTICE	B	Divide and conquer merges pairs of lists hierarchically across $\log(K)$ tree levels, performing optimal work vs K sequential passes.
Q20	PRACTICE	B	A fully skewed degenerate tree of N nodes has maximum height $N - 1$ edges.
Q21	PRACTICE	B	Key should be the element's numeric value so complement lookup (<code>target - num</code>) runs in $O(1)$ time, returning stored index as value.
Q22	PRACTICE	B	LCA in BST is the node where target values diverge (one strictly smaller, one strictly larger) or one matches the current node value.
Q23	PRACTICE	A	Array slicing copies sub-array memory allocations, introducing unexpected $O(N)$ extra space and time overhead on recursive calls.
Q24	PRACTICE	A	Passing pointer bounds <code>left</code> and <code>right</code> operates directly on existing memory without copying arrays.
Q25	PRACTICE	B	Heap insertion involves placing element at leaf and percolating up along tree height, which takes $O(\log N)$ time.